Transpareo

transpareo-md2pdf

Markdown to PDF, rendered through headless Chromium. Tables, code blocks and CSS behave the way they do on the web, because a browser is doing the layout.

Contents

Why a browser

Every layout problem a document generator has to solve has already been solved, repeatedly, by browser engines: column sizing, orphan and widow control, page breaks inside tables, font fallback, ligatures, bidirectional text. Rather than reimplement any of it, md2pdf hands Chromium a self-contained HTML document and asks it to print.

	md2pdf	**wkhtmltopdf**	**`pandoc --pdf`**
Layout engine	current Chromium	WebKit, ~2012	LaTeX
CSS support	whatever Chrome does	partial, dated	none
TOC page numbers	read from the PDF	manual	yes
External programs	Chromium	wkhtmltopdf	pandoc + TeX
Install size	browser only	~50 MB	~2 GB with TeX

Chromium is the only external program required. Markdown parsing, syntax highlighting and PDF inspection are gems, so `bundle install` covers them.

Install

```
gem install transpareo-md2pdf
md2pdf doctor
```

```
  ok    chromium      148.0.7778.215  /usr/bin/chromium
  ok    commonmarker  2.9.0  gem
  ok    nokogiri      1.19.4  gem
  ok    rouge         4.7.0  gem
  ok    pdf-reader    2.15.1  gem
  ok    rubyzip       2.4.1  gem
```

Most machines already have a browser. If yours does not, either install one with your package manager or let md2pdf fetch a known-good build:

```
md2pdf install-deps
```

That downloads Google's `chrome-headless-shell` for your platform into `~/.local/share/md2pdf`, verifies it against a SHA-256 sum shipped in the gem, and unpacks it. No root, no `PATH` changes, and it never runs on its own. `--force` reinstalls; `--latest` takes current stable instead of the pinned build, which skips checksum verification since there is nothing pinned to compare against.

Chromium is resolved from `CHROMIUM`, then the `install-deps` directory, then `PATH`, then the standard macOS app bundles.

Which Chromium you use changes the output

The browser lays the document out, so its version is part of the rendering. Two machines on different Chromium builds produce slightly different PDFs from the same markdown: a paragraph that fit on one page may run onto the next, and the page count with it. Nothing is wrong when that happens, and the stylesheet is not the cause.

That is the argument for `install-deps`. A pinned build renders identically everywhere it is installed, which matters when a PDF is a deliverable rather than a preview. **Note that installing it also changes what your machine renders with**, since the managed build outranks anything on `PATH`. `md2pdf doctor` prints the version and path actually in use.

To keep using the system browser after installing, name it:

```
CHROMIUM=/usr/bin/chromium md2pdf report.md
```

On a bare server

A downloaded Chromium is a binary, not an installation. Your distro's `chromium` package pulls in the shared libraries it needs; the archive from Google does not. So on a minimal server or a slim container, `install-deps` succeeds and the binary still cannot start:

```
chrome-headless-shell: error while loading shared libraries:
libXdamage.so.1: cannot open shared object file
```

`md2pdf doctor` detects this. It starts the binary rather than just checking the file is there, and lists **every** missing library, not only the first one the loader complains about, then names the packages that supply exactly those:

```
  MISS  chromium      cannot start, missing libXdamage.so.1, libXfixes.so.3

chromium: cannot start, missing libXdamage.so.1, libXfixes.so.3
  sudo apt install libxdamage1 libxfixes3
```

`install-deps` runs the same check after unpacking and fails loudly rather than reporting a success you cannot use. Add `--with-libraries` and it will show that command and offer to run it:

```
md2pdf install-deps --with-libraries        # asks before sudo
md2pdf install-deps --with-libraries --yes  # for a deploy script
```

Plain `install-deps` never uses sudo. It writes only into `~/.local/share/md2pdf`, and that stays true. Without a terminal to answer the prompt it prints the command and stops rather than blocking on a password nobody will see.

On Ubuntu, do not install the distro browser. There is no `chromium` package, and `chromium-browser` is a transitional shim that pulls in snapd and a confined browser, which cannot read the temporary files md2pdf hands it. Download the binary and add the two or three libraries it wants. On Debian, Fedora and Arch the distro `chromium` is fine and brings its own closure.

Usage

```
md2pdf file.md
md2pdf *.md                # all .md in current dir
md2pdf 'docs/*.md'         # globs are expanded internally
md2pdf                     # same as md2pdf *.md
md2pdf --flat file.md      # only H1 rendered as heading
md2pdf --unwrap file.md    # join hard-wrapped paragraphs
md2pdf --no-toc file.md    # skip the TOC
```

PDFs are written next to their inputs unless `--output` or `--output-dir` says otherwise.

Reading from a pipe

`-` means standard input, and it is assumed when nothing else is named and stdin is not a terminal:

```
cat report.md | md2pdf - > report.pdf
md2pdf < report.md > report.pdf
pandoc -t gfm notes.docx | md2pdf - --output=notes.pdf
./code-stats.py --locale=de | md2pdf - --locale=de > stats.pdf
```

The program on the left writes markdown to its own standard output; `|` hands that to md2pdf; `>` captures the PDF. Note the locale appearing twice in the last example: the first flag tells your generator which language to write, the second tells md2pdf which language to label the table of contents and footnotes in. They are different programs and neither can read the other's arguments. Alternatively have the generator emit front matter, or set `locale:` in `.md2pdf.yml`, and md2pdf picks it up.

With no destination the PDF goes to stdout, since there is no input filename to sit next to. Progress messages go to stderr in that mode so they cannot land inside the document. `--output` or `--output-dir` writes a file instead, and md2pdf refuses to spray PDF bytes at a terminal.

Front matter in piped input is honoured exactly as it is in a file. Relative image paths resolve against the working directory, and the config search starts there too. Since there is no filename, the locale cannot be auto-detected.

With `--output-dir` but no `--output`, the file is named after the document's first heading, reduced to something safe to put in a path: `# Q3/Q4 Results` yields `Q3-Q4-Results.pdf` rather than creating a `Q3/` directory. Pass `--output` when you want to choose the name yourself.

When a pipeline misbehaves

Reading a pipe blocks until the program upstream finishes and closes it, so md2pdf prints `reading markdown from stdin` as soon as it starts. If that line appears and nothing follows, the wait is in your generator rather than here. Run it on its own to confirm:

```
time ./code-stats.py --locale=de > /tmp/stats.md
```

Separating the two stages that way is also the quickest route to finding out which one produced a surprising document.

Use `set -o pipefail` in scripts. Without it a shell pipeline reports the exit status of the last command only, so a generator that dies halfway leaves you with a PDF of whatever it managed to print, and a successful exit status.

As a library

```ruby
require 'transpareo/md2pdf'

Transpareo::Md2pdf.convert('report.md', toc: true, locale: 'de')
```

Returns true when the PDF was written. Missing dependencies raise `MissingDependencyError`, a dependency that will not start raises `UnusableDependencyError`, render failures raise `ConversionError`, and nothing in the library calls `exit`. All inherit from `Transpareo::Md2pdf::Error`, so one rescue covers the lot.

Prefer this over shelling out if you are calling md2pdf from another application. The executable reports failures on stderr, which a host process typically discards or routes somewhere its own logger never sees, leaving you with an exit status and no cause. The exception carries the browser's own output in its message:

```ruby
begin
  Transpareo::Md2pdf.convert(path)
rescue Transpareo::Md2pdf::Error => e
  Rails.logger.error("md2pdf: #{e.class}: #{e.message}")
  raise
end
```

What it understands

Everything in GitHub Flavored Markdown, plus a few things beyond it.

Feature	Notes
Tables	Header words wrap individually so columns size sanely
Fenced code	Highlighted by Rouge, inline styles, self-contained
Images	Local files inlined, so the PDF has no external refs
Footnotes	Renumbered in reference order, duplicates merged
Task lists	`- [ ]` and `- [x]`
Strikethrough	`~~text~~`
Autolinks	Bare URLs become links

Feature	Notes
Fenced divs	`::: name` blocks become `<div class="name">`, for callouts and title-page control
Front matter	An `md2pdf:` YAML block, stripped before rendering

Images

Paths are resolved **relative to the markdown file**, not to your working directory, so a document renders the same wherever you run md2pdf from. Local images are read and embedded as data URIs, which means the finished PDF carries its own artwork and does not break when the source tree moves.

PNG, JPEG, GIF, SVG, WebP, AVIF, BMP and ICO are recognised. Remote `http(s)` sources are left for the browser to fetch, so they need network access at render time. A missing or unreadable image warns and is skipped rather than failing the whole document.

Fenced divs and callouts

Any `::: name` block becomes `<div class="name">` around whatever it contains, and the markdown inside is parsed normally. Pair that with `--custom-css` and you have callouts, admonitions, pull quotes, or anything else your stylesheet cares to define:

```
::: warning

**Do not** run this against production. The migration is not
reversible.

:::
```

```
/* passed with --custom-css */
.warning {
  border-left: 3px solid #c0392b;
  background: #fdf2f0;
  padding: 0.8em 1em;
  page-break-inside: avoid;
}
```

The class name is taken verbatim, so `::: note`, `::: tip` and `::: aside` all work without md2pdf knowing anything about them. Pandoc's brace form, `::: {.note}`, is accepted too.

Blocks nest, and an unclosed block is closed at the end of the document rather than swallowing the rest of it.

`::: intro` is the one name md2pdf treats specially: it suppresses the centered title page, so the introduction flows on page 1 right after the title. See Title page.

A `:::` inside a fenced code block is left alone, so documentation about fenced divs survives being documented.

Table of contents

A TOC covering H2 and H3 is inserted by default on its own page, between the title page and the first H2, carrying **real page numbers**. It is skipped for short documents: both `--toc-min` (default 3 H2s) and `--toc-min-words` (default 1500) must be met.

The numbers are not estimated. The document is rendered once, Chromium writes a PDF destination for every heading the TOC links to, and the second pass reads that table back and bakes the numbers in. Because they come from the PDF's own structure rather than from scraping extracted text, they stay correct even when a heading lands right at a page boundary. This roughly doubles render time.

Title page

The H1 and its lead block are vertically centered on page 1 **only when the document has a TOC and no `::: intro :::` block**. Otherwise the title flows from the top of page 1.

Use `::: intro :::` to keep an introduction on page 1, right after the title:

```
# Document Title

*Subtitle line.*

::: intro

A multi-paragraph introduction that lands on page 1, right after
the subtitle, before the TOC starts on the following page.

:::

## First Section
```

Footnotes

```
The first attempt failed.[^attempt]
A later attempt succeeded.[^attempt]

[^attempt]: Internal lab note, 2026-04-19.
```

Each `[^id]` becomes a numbered superscript link, numbered in the order references appear. Entries whose definition text is identical are merged into one item, which is what you want when the same source is cited from several places. Every entry links back to its first reference.

A trailing `## Footnotes` heading in the source is consumed so it does not double up with the rendered one. The heading is set with `--footnotes-label="Sources"`, or omitted entirely with `--footnotes-label=""`.

Options reference

Every flag, with the config key that sets the same thing. Flags beat front matter, front matter beats `.md2pdf.yml`.

Commands

Command	Description
`doctor`	Report dependency status and exit non-zero if any are missing
`install-deps`	Download a pinned, checksum-verified Chromium
`install-deps --latest`	Take current stable instead of the pinned build; skips checksum verification
`install-deps --force`	Reinstall even if that version is already present

Content

Flag	Config key	Default	Description
`--flat`	`flat`	`false`	Demote H2/H3 to bold paragraphs, leaving H1 the only heading

Flag	Config key	Default	Description
`--single-heading`	`flat`	`false`	Alias for `--flat`
`--unwrap`	`unwrap`	`false`	Join hard-wrapped paragraph lines back into one
`--toc`	`toc`	on	Force a TOC, ignoring the auto-skip thresholds below
`--no-toc`	`toc: false`		Disable the TOC entirely
`--toc-depth=N`	`toc-depth`	`2`	`1` = H2, `2` = H2 + H3, `3` = also H4
`--toc-label=TEXT`	`toc-label`	per locale	TOC heading text
`--toc-min=N`	`toc-min`	`3`	H2s required before a TOC appears
`--toc-min-words=N`	`toc-min-words`	`1500`	Words required before a TOC appears
`--footnotes-label=TEXT`	`footnotes-label`	per locale	Footnotes heading; `""` renders the list with no heading
`--locale=CODE`	`locale`	auto	Sets default labels and the `lang` attribute
	`locales`		Custom label sets, see below. Config only

Both `--toc-min` and `--toc-min-words` must be satisfied for a TOC to appear automatically. `--toc` overrides both.

Typography

Flag	Config key	Default
`--font-size=Npt`	`font-size`	`11pt`
`--line-height=N`	`line-height`	`1.8`
`--font-family="..."`	`font-family`	Plus Jakarta Sans, DejaVu Sans, Helvetica
`--code-font-family="..."`	`code-font-family`	JetBrains Mono, DejaVu Sans Mono, Menlo

Fonts must be installed on the rendering machine. Chromium falls back silently if one is missing, so check the output when using a font the machine may not have.

Layout

Flag	Config key	Default
`--page-size=NAME`	`page-size`	`A4`. Any CSS page size: `Letter`, `A5`, `Legal`
`--margins="T R B L"`	`margins`	`22mm 20mm 24mm 20mm`, in CSS order
`--output=FILE`	`output`	`<input>.pdf`
`--output-dir=DIR`	`output-dir`	alongside the input

A relative `output-dir` in a config file resolves against that config file, so `output-dir: build` collects PDFs in the project's `build/` no matter which directory you run from. As a flag it resolves against your shell. md2pdf prints the absolute path of each file it writes.

Branding

Flag	Config key	Default	Description
`--logo=PATH`	`logo`	none	SVG logo. The footer version is derived from the same file in grey, so one asset covers both
`--footer-title=TEXT`	`footer-title`	the document's H1	Running text centred in the footer, between the logo and the page number. `""` removes it
`--no-header-logo`	`header-logo: false`	shown	Hide the first-page logo
`--no-footer-logo`	`footer-logo: false`	shown	Hide the per-page footer logo
`--no-page-numbers`	`page-numbers: false`	shown	Hide the page counter

The footer has three slots: the logo on the left, the title in the centre, and the page counter on the right. Each is independent, so any combination works.

The centre slot carries the document's own title by default, read from its first H1 after parsing, so a setext heading works and inline markup is reduced to plain text. Override it when the heading is too long for a running footer, or when the PDF should carry a project name rather than a document name:

```
footer-title: Quarterly Platform Report
```

Pass an empty string to remove it, which also matches the previous behaviour of no centre slot at all:

```
md2pdf --footer-title="" report.md
```

A document with no H1 gets no footer title. Keep overrides short: the centre slot sizes to its content, so a long title runs into the logo or the page number rather than wrapping or truncating.

md2pdf ships no logo and renders unbranded out of the box. The Transpareo mark on the sample pages above is exactly that: a sample, supplied to the renderer the same way you would supply your own. It lives in this repository for the demo and is deliberately excluded from the published gem, so installing md2pdf gives you a blank canvas rather than somebody else's branding.

See Using your own logo below.

Style

Flag	Config key	Default	Description
`--link-color=#hex`	`link-color`	`#0a4a90`	Anchor colour throughout
`--custom-css=FILE`	`custom-css`	none	Appended to the stylesheet, so it wins on ties

Misc

Flag	Description
`--open`	Open the PDF in your default viewer afterwards, using `open` on macOS, `xdg-open` on Linux and `start` on Windows. Rejected when more than one file would be converted
`-v`, `--version`	Print the version and exit
`-h`, `--help`	Print help and exit

Exit codes: `0` success, `1` a render or dependency failure, `2` a usage error, `130` interrupted.

Configuration

Settings come from three places, highest priority first.

1. CLI flags.

2. YAML front matter, under an `md2pdf:` key so it cannot collide with anything else:

```
---
md2pdf:
  font-size: 14pt
  toc-label: Inhalt
---

# My Document
```

The block is stripped before rendering, so it never appears in the PDF.

3. `.md2pdf.yml` files, every one between the document and `$HOME`, nearer files layering over further ones:

```
font-size: 14pt
line-height: 1.6
page-size: A4
margins: "22mm 20mm 24mm 20mm"
logo: assets/brand/logo.svg
link-color: "#0a4a90"
page-numbers: false
```

Keys mirror the CLI flags without the `--` prefix. Negating flags invert: `--no-page-numbers` becomes `page-numbers: false`.

4. A global config, applying to every document you render: `$XDG_CONFIG_HOME/md2pdf/config.yml` (usually `~/.config/md2pdf/config.yml`), or `~/.md2pdf.yml` if that is absent. Unlike the walk above, this applies wherever the document lives, including outside `$HOME`.

How the layers combine

Every applicable config is read and merged key by key, least specific first:

```
~/.config/md2pdf/config.yml     logo, fonts, link colour
  ~/work/.md2pdf.yml            page size for everything at work
    ~/work/handbook/.md2pdf.yml  a footer title for this one book
      front matter                one document's own overrides
        command-line flags         this invocation only
```

A file only has to state what it changes. Setting `logo` globally and `font-size` in a project keeps both: the project file adds to what it inherits rather than replacing it. `locales:` merges the same way, so a project can adjust one label without discarding the rest of the table.

That means one file at the root of a docs tree brands and styles everything beneath it, and committing `.md2pdf.yml` next to your docs gives every contributor identical PDFs without any flags. Relative paths in each file resolve against that file, so a config can point at assets beside itself no matter where it is run from.

To see which settings a document ends up with, render it and look, or read the files in the order above. There is no precedence subtlety beyond nearer-wins.

Using your own logo

Point `--logo` at an SVG, or set `logo:` in `.md2pdf.yml` so every document under that directory picks it up:

```yaml
logo: assets/brand/logo.svg
```

A relative path is relative to whatever declared it. A path in `.md2pdf.yml` resolves against that config file, one in a document's front matter resolves against the document, and one given as a flag resolves against your shell, which is what typing a path into a terminal implies. The rule covers every setting naming a file or directory: `logo`, `custom-css`, `output` and `output-dir`.

That means a repository can commit its branding next to its docs and every contributor produces identically branded PDFs from any working directory, without passing a flag. `MD2PDF_LOGO` sets a per-machine default when the asset lives outside the project.

One asset covers both placements. The header logo on page 1 uses your original colours. The small footer logo repeated on every page is derived from the same file by rewriting each fill, gradient stop and inline `fill:` to grey, so there is no second file to keep in sync.

What makes a good logo file here:

- **SVG.** It is the only format the recolouring understands, and it stays sharp at any zoom. Raster formats work in document body images, not as the logo.
- **A wide wordmark.** The header box is 42mm by 6.95mm and the footer is 18mm by 2.98mm, both roughly 6:1. A square or tall mark will be squashed, so crop the artwork or set your own sizes with `--custom-css`.
- **Colour defined as fills or gradient stops**, not embedded raster or CSS classes, or the grey footer derivation will not find anything to recolour.

If the file is missing, md2pdf warns and renders without it rather than failing the document. Use `--no-header-logo` or `--no-footer-logo` to suppress either placement.

The sample logo in this repository is Transpareo's trademark and is present only to demonstrate the feature. The MIT licence covers the code, not the brand assets. Replace it with your own.

Custom localizations

The built-in label sets cover `en`, `de`, `fr`, `es`, `it`, `pt` and `nl`. A `locales:` block overrides any of them and adds locales that are not built in:

```yaml
logo: assets/brand/logo.svg

locales:
  de:
    toc-label: Inhaltsverzeichnis   # override a built-in
  sv:
    toc-label: Innehåll             # add a new one
    footnotes-label: Fotnoter
```

An entry only has to name the labels it changes; anything omitted keeps its built-in wording. Once a locale is defined here, filename detection recognises it too, so `handbok.sv.md` picks up the Swedish labels and gets `lang="sv"` on the output.

The document itself still wins, so a single file can opt out:

```
---
md2pdf:
  toc-label: Agenda
---
```

Environment

```
CHROMIUM=/usr/bin/chromium          Browser override
MD2PDF_LOGO=/path/to/logo.svg       Default for --logo
MD2PDF_HOME=~/.local/share/md2pdf   Managed install directory
MD2PDF_OPENER=zathura               Viewer used by --open
```

`MD2PDF_OPENER` overrides the per-platform default and may carry arguments, as in `MD2PDF_OPENER="flatpak run org.gnome.Evince"`. It must name a real program: a shell alias or function is invisible to a spawned process, so aliasing `open` in your shell will not reach md2pdf.

Docker

```
docker build -t transpareo-md2pdf .
docker run --rm -v "$PWD:/work" transpareo-md2pdf report.md
```

The image carries Chromium and the fonts it needs.

Continuous integration

GitHub's Ubuntu runners already ship Chrome, which md2pdf finds on `PATH`, so usually nothing extra is needed:

```
- uses: ruby/setup-ruby@v1
  with:
    ruby-version: '3.3'
    bundler-cache: true

- run: bundle exec md2pdf docs/*.md
```

Avoid `apt-get install chromium-browser` on Ubuntu runners: it is a snap transitional package that does not run headless. On images with no browser at all, `bundle exec md2pdf install-deps` caches well, since the download URL is pinned.

Contributing

See CONTRIBUTING.md for the architecture, the filter pipeline and the release process.

License

MIT. See LICENSE.txt.

The licence covers the source. `docs/assets/transpareo-logo.svg` is a trademark of Transpareo, included solely as a worked example of the branding options, and is not part of the distributed gem.